ПАРАЛЛЕЛЬНЫЕ АЛГОРИТМЫ ПОИСКА В ШИРИНУ В ГРАФЕ

Поиск в ширину (BFS – Breadth-First Search) является одним из методов обхода графа. Поиск в графах играет важную роль в анализе больших наборов данных. Поиск в ширину можно использовать для нахождения кратчайших путей между вершинами, для определения двудольности связной компоненты графа, для выделения компонент связности неориентированного графа, а также для поиска множества путей между двумя вершинами, длина которых лежит в заданном диапазоне.

BFS входит в тестовый комплект проекта Graph 500. Список Graph 500 содержит рейтинги (оценку производительности) суперкомпьютеров, связанных с задачами по обработке очень больших массивов данных.

Последовательный алгоритм

Пусть $G(V,E)$ – граф, V – множество вершин, E – множество ребер. Будем считать, что вершины графа занумерованы последовательными целыми числами, начиная от 0 и заканчивая n–1, а граф задан матрицей смежности A размера $n \times n$, где $a_{u,v}=1$, если существует ребро (дуга в случае орграфа), идущая из вершины u в вершину v, и $a_{u,v}=0$ в противном случае.

Приведем последовательный алгоритм BFS [1].

Вход: граф $G(V,E)$, стартовая вершина v_s.

1. Присвоить стартовой вершине v_s метку (уровень) $L(v_s)=0$, занести ее в очередь и считать помеченной.

2. Повторить следующую последовательность действий, пока все вершины не помечены и очередь не станет пустой. Пусть первым элементом очереди является вершина u и $Q(u)$ – множество непомеченных вершин, смежных с вершиной u; тогда

а) каждую вершину $v \in Q(u)$:

пометить меткой (уровнем) $L(v)=L(u)+1$,

запомнить, что вершина v была помечена из вершины u: $P(v)=u$,

занести v в очередь;

б) вершину u считать просмотренной, удалить ее из очереди и вернуться к шагу 2 алгоритма.

Выход: массив вершин $L[1..n]$, где $L(v)$ – расстояние между вершинами v и v_s (уровень v); массив вершин $P[1..n]$, где $P(v)$ – вершина, из которой была помечена вершина v (родитель v).

Рассмотрим пример. Пусть орграф имеет четыре вершины, $v_s=0$, задана матрица смежности

$$A=\begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

Помечаем стартовую вершину: присваиваем $L(v_S)=L(0)=0$; заносим вершину 0 в очередь F: $F=\{0\}$.

Просматриваем вершину 0. Помечаем вершины множества $Q(0)=\{1, 3\}$: $L(1)=1$, $P(1)=0$, $L(3)=1$, $P(3)=0$; заносим эти вершины в очередь, вершину 0 из очереди удаляем: $F=\{1, 3\}$.

Просматриваем вершины 1 и 3. Помечаем вершину множества $Q(1)=\{2\}$: $L(2)=2$, $P(2)=1$; заносим вершину 2 в очередь. Множество $Q(3)$ пустое, так как смежная с вершиной 3 вершина 1 уже помечена. Вершины 1 и 3 из очереди удаляем. Теперь $F=\{2\}$.

Просматриваем вершину 2. Эта вершина не имеет смежных вершин, поэтому $Q(2)=\varnothing$, все вершины помечены, $F=\varnothing$.

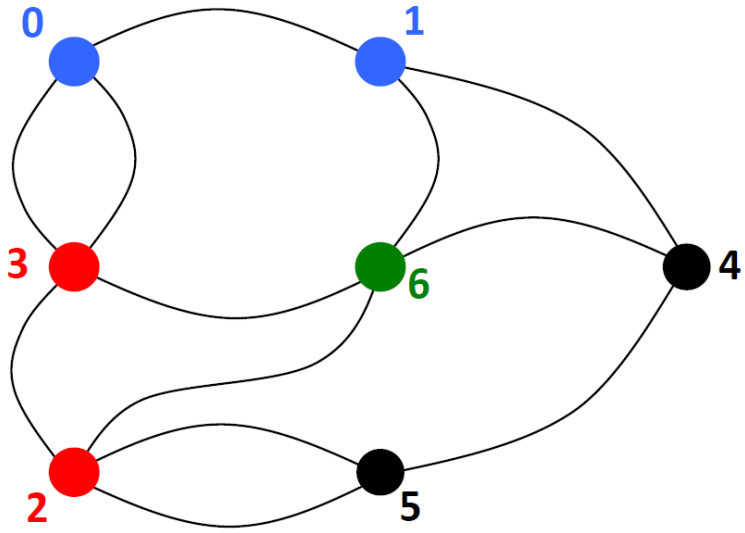
Параллельный 1D алгоритм для реализации на компьютере с распределенной памятью

Поиск по очень большим (десятки миллиардов вершин и ребер) графам вызывает затруднение из-за огромного пространства для поиска: невозможно сохранить настолько большие графы в оперативной памяти одного компьютера. Возникает необходимость в разработке распределенных (т.е. предназначенных для распределенных вычислений) параллельных алгоритмов поиска в ширину.

Рассмотрим распределенный алгоритм поиска в ширину, использующий одномерное (1D) разбиение графа.

1D разбиение графа – такое разбиение, при котором вершины каким-то образом распределены между процессами, а ребра, выходящие из некоторой вершины, принадлежат тому же процессу, что и данная вершина. Вершины, принадлежащие некоторому процессу, называются его локальными вершинами.

Приведем пример 1D разбиения графа, изображенного на рисунке, для четырех процессов. Пусть вершины 0 и 1 принадлежат нулевому процессу, вершины 2 и 3 принадлежат первому процессу, вершины 4 и 5 принадлежат второму процессу, вершина 6 принадлежит третьему процессу.



Тогда ребра делятся между процессами следующим образом:

(0,1) (0,3) (0,3) (1,0) (1,4) (1,6)
(2,3) (2,5) (2,5) (2,6) (3,0) (3,0) (3,2) (3,6)
(4,1) (4,5) (4,6) (5,2) (5,2) (5,4)
(6,1) (6,2) (6,3) (6,4)

Пусть некоторый граф задан матрицей смежности A. Будем считать, не нарушая общности, что строки и столбцы матрицы A симметрично переупорядочены так, чтобы вершины, принадлежащие одному процессу, были приписаны соседним строкам и столбцам матрицы (например, строки 1, 2 и столбцы 1, 2 соответствуют вершинам процесса 0, а строки 3, 4 и столбцы 3, 4 – вершинам процесса 1). Тогда матрица смежности A имеет вид

$$\begin{pmatrix} A_1 \\ A_2 \\ \dots \\ A_P \end{pmatrix},$$

где A_i есть блок из строк матрицы смежности, принадлежащих процессу i–1.

Ребра, исходящие из вершины v, формируют строку с номером v матрицы смежности; строка v формирует список смежности вершины v. Ребра, входящие в этот список, принадлежат тому же процессу, что и вершина v. Для балансировки разбиения, каждый процесс должен владеть примерно одинаковыми количествами вершин и исходящих из них ребер.

Распределенный поиск в ширину с 1D разбиением выполняется следующим образом. На итерации l алгоритма просматриваются вершины уровня l; помечаются вершины уровня l на предыдущей, (l–1)-й итерации (стартовая вершина помечается при инициализации). Пусть F – множество вершин уровня l, принадлежащих данному процессу. Из вершин, смежных вершинам из F, формируют множество N соседних вершин. Некоторые из этих вершин могут принадлежать данному процессу, а некоторые – не принадлежать. Во втором случае эти вершины посылаются соответствующим процессам (чтобы быть помеченными и оказаться во множествах F этих процессов на следующем уровне). Каждый процесс получает эти (посланные процессу из множеств N других процессов) вершины; уже помеченные вершины далее не рассматривает. Каждая вершина v помечается уровнем $L(v)$, который обозначает расстояние между вершинами v и v_s, а также помечается родителем $P(v)$ вершины v.

Вершины распределены между процессами. Структуры данных $L(v)$ и $P(v)$ распределены: каждый процесс хранит данные лишь о своих вершинах. Информация о распределении вершин между процессами хранится каждым процессом. Информацию, что вершина является стартовой, знает процесс, которому эта вершина распределена.

Алгоритм 1 (параллельный алгоритм BFS для распределенных вычислений с 1D разбиением графа) [2].

Вход: распределение вершин между процессами (процессы знают номера своих вершин и номера вершин любого другого процесса), матрица смежности A (процесс с номером i владеет блоком A_{i+1}), стартовая вершина v_S (процесс, которому распределена стартовая вершина, знает её номер).

Выход: $L(v)$ – массив уровней вершин, $P(v)$ – массив родителей вершин (каждый процесс хранит данные о назначенных ему вершинах).

Для каждого процесса с номером i:

1: Инициализация: $L(v)=0$, если $v=v_S$; $L(v)=\infty$, если $v \neq v_S$

2: for $l = 0$ to ∞ do

3: Сформировать $F=\{u \mid L(u)=l\}$ // множество локальных вершин уровня l (вершины, помеченные на предыдущей итерации на шаге 13 или стартовая вершина на нулевой итерации)

4: if для всех процессов $F=\varnothing$ then

5: завершить все вычислительные процессы

endif

6: Сформировать N – множество соседей вершин из F, пользуясь своими строками из матрицы A, т.е. строками блоков A_{i+1}; для каждого v из N следует указывать родителя, т.е. N формировать из пар (v,u), где u – родитель v (из пар (v,u_1) и (v,u_2) включается пара с меньшим значением u)

7: for all processes q do

8: Сформировать N_q – множество вершин из N, распределенных процессу q // N_q могут быть пустыми

9: Send N_q to process q; себе (случай $q=i$) N_q не высылается

10: Receive $\overline{N}_q$ from process q, $\overline{N}_q$ – вершины текущего процесса, принадлежащие множеству N процесса q

endfor

11: Сформировать множество $\overline{N}=\bigcup_q \overline{N}_q$ // $\overline{N}_q$ могут пересекаться: дуги в разных процессах могут привести в одинаковые вершины; $\overline{N}$ формируется из пар (v,u) (из (v,u_1) и (v,u_2) включается пара с меньшим значением u); $\overline{N}_q$ могут иметь помеченные вершины;

12: for $v \in \overline{N}$ and $L(v)=\infty$ do // цикл по всем непомеченным вершинам из вновь сформированного множества локальных вершин

13: $L(v)=l+1$, $P(v)=u$ // N и $\overline{N}$ формировались из пар (v,u)

endfor

endfor(l)

Пример применения параллельного Алгоритма 1. Пусть граф задан матрицей смежности:

$$A=\begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

Граф с такой матрицей смежности является одним из вариантов ориентированного графа, полученного на основе рассмотренного ранее не ориентированного графа.

Пусть v_S=0, число процессов – 4, вершины 0 и 1 принадлежат нулевому процессу, вершины 2 и 3 принадлежат первому процессу, вершины 4 и 5 принадлежат второму процессу, вершина 6 принадлежит третьему процессу.

Разбиение опишем схемой:

		Pr_0		Pr_1		Pr_2		Pr_3
	v	0	1	2	3	4	5	6
Pr_0	0	0	1	0	1	0	0	0
	1	0	0	0	0	1	0	1
Pr_1	2	0	0	0	0	0	1	1
	3	1	0	1	0	0	0	1
Pr_2	4	0	0	0	0	0	1	0
	5	0	0	1	0	0	0	0
Pr_3	6	0	0	0	0	1	0	0

Применим параллельный Алгоритм 1.

Инициализация:

Pr_0 : $L(v_S)=L(0)=0$, $L(1)=\infty$,

Pr_1 : $L(2)=\infty$, $L(3)=\infty$,

Pr_2 : $L(4)=\infty$, $L(5)=\infty$,

Pr_3 : $L(6)=\infty$.

Итерация l = 0:

Множества F:

Pr_0 : $F=\{0\}$,

Pr_1 , Pr_2 , Pr_3 : $F=\varnothing$.

Множества N (N формируем из пар (v,u), где u – родитель v),

N_q (N_q указаны только не пустые):

Pr_0 : $N=\{(1,0),(3,0)\}$, $N_0=\{(1,0)\}$, $N_1=\{(3,0)\}$,

Pr_1 , Pr_2 , Pr_3 : $N=\varnothing$.

Результаты обмена множествами N_q:

Pr_0 : $\overline{N}_0=\{(1,0)\}$, $\overline{N}=\{(1,0)\}$,

Pr_1 : $\overline{N}_0=\{(3,0)\}$, $\overline{N}=\{(3,0)\}$,

Pr_2 , Pr_3 : $\overline{N}=\varnothing$.

Метки $L(v)$ и $P(v)$ для $v\in\overline{N}$ (если $L(v)=\infty$):

Pr_0 : $L(1)=1$, $P(1)=0$,

Pr_1 : $L(3)=1$, $P(3)=0$,

Pr_2 , Pr_3 : – .

Итерация l = 1:

Множества F:

Pr_0 : $F=\{1\}$,

Pr_1 : $F=\{3\}$,

Pr_2 , Pr_3 : $F=\varnothing$.

Множества N, N_q:

Pr_0 : $N=\{(4,1),(6,1)\}$, $N_2=\{(4,1)\}$, $N_3=\{(6,1)\}$,

Pr_1 : $N=\{(0,3),(2,3),(6,3)\}$, $N_0=\{(0,3)\}$, $N_1=\{(2,3)\}$, $N_3=\{(6,3)\}$,

Pr_2 , Pr_3 : $N=\varnothing$.

Результаты обмена множествами N_q:

Pr_0 : $\overline{N}_1=\{(0,3)\}$, $\overline{N}=\{(0,3)\}$,

Pr_1 : $\overline{N}_1=\{(2,3)\}$, $\overline{N}=\{(2,3)\}$,

Pr_2 : $\overline{N}_0=\{(4,1)\}$, $\overline{N}=\{(4,1)\}$,

Pr_3 : $\overline{N}_0=\{(6,1)\}$, $\overline{N}_1=\{(6,3)\}$, $\overline{N}=\{(6,1)\}$, ((v,u)=(6,3) в $\overline{N}$ не вошло, так как v=6 уже присутствует в паре (v,u)=(6,1).

Метки $L(v)$ и $P(v)$ для $v\in\overline{N}$ (если было $L(v)=\infty$):

Pr_0 : – (вершина 0 уже помечена: $L(0)=0$),

Pr_1 : $L(2)=2$, $P(2)=3$,

Pr_2 : $L(4)=2$, $P(4)=1$,

Pr_3 : $L(6)=2$, $P(6)=1$.

Итерация l = 2:

Множества F:

Pr_0 : $F=\varnothing$,

Pr_1 : $F=\{2\}$,

Pr_2 : $F=\{4\}$,

Pr_3 : $F=\{6\}$.

Множества N, N_q:

Pr_0 : $N=\varnothing$,

Pr_1 : $N=\{(6,2),(5,2)\}$, $N_2=\{(5,2)\}$, $N_3=\{(6,2)\}$,

Pr_2 : $N=\{(5,4)\}$, $N_2=\{(5,4)\}$,

Pr_3 : $N=\{(4,6)\}$, $N_2=\{(4,6)\}$.

Результаты обмена множествами N_q:

Pr_0 : $\overline{N}=\varnothing$,

Pr_1 : $\overline{N}=\varnothing$,

Pr_2 : $\overline{N}_1=\{(5,2)\}$, $\overline{N}_2=\{(5,4)\}$, $\overline{N}_3=\{(4,6)\}$, $\overline{N}=\{(5,2),(4,6)\}$,

((v,u)= (5,4) в $\overline{N}$ не вошло, так как v=5 уже присутствует в паре (v,u)=(5,2)),

Pr_3 : $\overline{N}_1=\{(6,2)\}$, $\overline{N}=\{(6,2)\}$.

Метки $L(v)$ и $P(v)$ для $v\in\overline{N}$ (если $L(v)=\infty$):

Pr_0 : – ,

Pr_1 : – ,

Pr_2 : $L(5)=3$, $P(5)=2$, (вершина 4 уже помечена),

Pr_3 : – (вершина 6 уже помечена).

Итерация l = 3:

Множества F:

Pr_0 : $F=\varnothing$,

Pr_1 : $F=\varnothing$,

Pr_2 : $F=\{5\}$,

Pr_3 : $F=\varnothing$.

Множества N, N_q:

Pr_0 : $N=\varnothing$,

Pr_1 : $N=\varnothing$,

Pr_2 : $N=\{(2,5)\}$, $N_1=\{(2,5)\}$,

Pr_3 : $N=\varnothing$.

Результаты обмена множествами N_q:

Pr_0: $\overline{N} = \emptyset$,

Pr_1: $\overline{N}_2 = \{(2,5)\}$, $\overline{N} = \{(2,5)\}$,

Pr_2: $\overline{N} = \emptyset$,

Pr_3: $\overline{N} = \emptyset$.

Метки $L(v)$ и $P(v)$ для $v \in \overline{N}$ (если $L(v) = \infty$):

Pr_0: – ,

Pr_1: – (вершина 2 уже помечена),

Pr_2: – ,

Pr_3: – .

Итерация l = 4:

Множества F: для всех процессов $F = \varnothing$,

все вычислительные процессы завершаются.

Параллельный 2D алгоритм для реализации на компьютере с распределенной памятью

Для оптимизации использования памяти и межпроцессорных коммуникаций вместо 1D разбиения графа используется двумерное (2D) разбиение. При 1D разбиении количество процессов, участвующих в групповых коммуникациях, равно $O(P)$, где P – общее количество процессов. При 2D разбиении количество процессов, вовлеченных в групповые коммуникации, равно $O\left(\sqrt{P}\right)$.

2D разбиение принято описывать посредством разбиения транспонированной матрицы смежности A^T на блоки. Блоки распределяются между процессами, так что каждая дуга принадлежит только одному процессу; кроме того, вершины тоже распределены между процессами.

Будем считать, что P процессов логически организованы в виде двумерного массива процессов размера $R \times C$. Процесс с координатами (i,j) обозначим $\mathrm{Pr}_{i,j}$ (нумерация i и j начинается с единицы). Пусть число $\frac{n_{adj}}{R \cdot C}$ является целым, где n_{adj} есть порядок матрицы смежности; если это не так, то можно добавить в граф фиктивные вершины.

Разбиение, которое мы будем использовать, опишем следующей схемой:

$$
\begin{array}{c}
 & \begin{matrix} \mathrm{Pr}_{1,1}-\mathrm{Pr}_{R,1} & \mathrm{Pr}_{1,2}-\mathrm{Pr}_{R,2} & \cdots & \mathrm{Pr}_{1,C}-\mathrm{Pr}_{R,C} \end{matrix} \\
\begin{matrix} \mathrm{Pr}_{1,1} \\ \mathrm{Pr}_{2,1} \\ \vdots \\ \mathrm{Pr}_{R,1} \\ \cdots \\ \mathrm{Pr}_{1,C} \\ \mathrm{Pr}_{2,C} \\ \vdots \\ \mathrm{Pr}_{R,C} \end{matrix} &
\begin{pmatrix}
A_{1,1}^{(1)} & A_{1,2}^{(1)} & \cdots & A_{1,C}^{(1)} \\
A_{2,1}^{(1)} & A_{2,2}^{(1)} & \cdots & A_{2,C}^{(1)} \\
\vdots & \vdots & \vdots & \vdots \\
A_{R,1}^{(1)} & A_{R,2}^{(1)} & \cdots & A_{R,C}^{(1)} \\
\cdots & \cdots & \cdots & \cdots \\
A_{1,1}^{(C)} & A_{1,2}^{(C)} & \cdots & A_{1,C}^{(C)} \\
A_{2,1}^{(C)} & A_{2,2}^{(C)} & \cdots & A_{2,C}^{(C)} \\
\vdots & \vdots & \vdots & \vdots \\
A_{R,1}^{(C)} & A_{R,2}^{(C)} & \cdots & A_{R,C}^{(C)}
\end{pmatrix}.
\end{array}
$$

Транспонированная матрица смежности A^T разбивается на $R \times C$ блочных строк и C блочных столбцов. Вершины распределяются между процессами следующим образом: процесс $\mathrm{Pr}_{i,j}$ владеет вершинами, соответствующими блочной строке с номером $(j-1)R+i$. Каждый процесс $\mathrm{Pr}_{i,j}$ обладает информацией о распределении вершин между процессами строки i.

Блок транспонированной матрицы смежности, обозначаемый $A_{i,j}^{(*)}$, принадлежит процессу $\mathrm{Pr}_{i,j}$. В каждом блоке $\frac{n_{adj}}{R \cdot C}$ строк и $\frac{n_{adj}}{C}$ столбцов. Один процесс владеет C блоками. Владение процесса блоком означает владение дугами, принадлежащими этому блоку, т.е. дугами, которые соответствуют ненулевым элементам этого блока транспонированной матрицы смежности.

Таким образом, каждый процесс владеет частичными списками смежности вершин, соответствующих входящим в блоки столбцам матрицы смежности. Суммарно процессы одного столбца владеют полными списками смежности принадлежащих им вершин.

Разбиения сбалансировано, если процессы владеют примерно одинаковым количеством ребер.

Распределенный поиск в ширину, использующий описанное 2D разбиение, выполняется следующим образом. Как и в случае использования 1D разбиения, на итерации l алгоритма просматриваются вершины уровня l (помечаются вершины уровня l на предыдущей, $(l-1)$-й итерации). Пусть F – множество вершин уровня l, принадлежащих данному процессу. Рассмотрим вершину u из множества F.

Процесс-владелец вершины u посылает процессам из этого же столбца массива процессов сообщение о том, что вершина u является вершиной текущего уровня. Каждый из этих процессов содержит свою часть списков смежности для этой вершины. Назовем эту операцию *expand*. Каждый из процессов в этом столбце процессов в результате таких коммуникационных операций получает множество вершин, находит для этих вершин соседние

вершины с использованием своих списков смежности и формирует множество N потенциальных вершин следующего уровня.

Обмены в пределах процессорного столбца необходимы, так как обладатель информации о наличии дуги из u в v («посредник» между u и v) может и не знать, что вершина u просматривается.

Вершины из N затем отправляются их владельцам, которые могут находиться только в той же строке массива процессов; эту операцию назовем *fold*. Каждый процесс получает посланные ему вершины из множеств N других процессов; уже помеченные вершины далее не рассматривают (заметим, что операции 8–11 Алгоритма 1 соответствуют операции *fold*). Каждая вершина v помечается уровнем $L(v)$, который обозначает расстояние между вершинами v и v_s, а также родителем $P(v)$ вершины v.

В операциях *expand* и *fold* заключается оптимизация по сравнению с 1D разбиением: в каждой операции группового обмена участвует лишь строка или столбец массива процессов, а не все процессы.

Структура данных $L(v)$ и структура данных $P(v)$ распределены: каждый процесс хранит данные лишь о своих вершинах. Каждый процесс обладает информацией о распределении вершин между процессами своей процессорной строки. Информацию, что вершина является стартовой, знает процесс, которому эта вершина распределена.

Алгоритм 2 (параллельный алгоритм BFS для распределенных вычислений с 2D разбиением графа) [2].

Вход: транспонированная матрица смежности A^T, информация о распределении вершин между процессами в пределах процессорной строки, стартовая вершина v_s (процесс, которому распределена стартовая вершина, знает её номер).

Выход: $L(v)$ – массив уровней вершин, $P(v)$ – массив родителей вершин.

Для каждого процесса $Pr_{i,j}$:

1: Инициализация: $L(v)=0$, если $v=v_s$; $L(v)=\infty$, если $v \neq v_s$

2: for $l = 0$ to ∞ do

3: Сформировать $F = \{u \mid L(u) = l\}$ // множество локальных вершин уровня l (вершины, помеченные на предыдущей итерации на шаге 17 или стартовая вершина на нулевой итерации)

4: if для всех процессов $F=\varnothing$ then

5: завершить все вычислительные процессы
endif

6: for all processes q in this processes column do

7: Send F to process q

8: Receive $\overline{F}_q$ from process q // $\overline{F}_q$ – это множество F процесса q
endfor

9: Сформировать множество $\overline{F} = \bigcup_q \overline{F}_q$

10: Сформировать N – множество соседей вершин из $\overline{F}$,

пользуясь своими списками смежности;
для каждого v из N следует указывать родителя,
т.е. N формировать из пар (v,u), где u – родитель v
(из пар (v,u_1) и (v,u_2) включается пара с меньшим значением u)

11: for all processes q in this processes row do

12: Сформировать N_q – множество вершин из N,
распределенных процессу q // N_q могут быть пустыми

13: Send N_q to process q; себе (случай $q=(i,j)$) N_q не высылается

14: Receive $\overline{N}_q$ from process q // $\overline{N}_q$ – вершины текущего
процесса, принадлежащие множеству N процесса q
endfor

15: Сформировать множество $\overline{N} = \bigcup_q \overline{N}_q$ // $\overline{N}_q$ могут пересекаться:
дуги в разных процессах могут привести в одинаковые вершины;
$\overline{N}$ формируется из пар (v,u), из пар (v,u_1) и (v,u_2) включается пара с меньшим значением u; $\overline{N}_q$ могут иметь помеченные вершины;

16: for $v \in \overline{N}$ and $L(v)=\infty$ do // цикл по всем непомеченным вершинам
из вновь сформированного множества локальных вершин

17: $L(v)=l+1$, $P(v)=u$ // N и $\overline{N}$ формировались из пар (v,u)
endfor
endfor(l)

Операции 6–9 Алгоритма 2 соответствуют операции *expand*, операции 11–15 соответствуют операции *fold*.

Отметим, что каждое множество F посылается всем процессам столбца массива процессов, а множество N_q – конкретному процессу строки массива.

Пример применения параллельного Алгоритма 2. Рассмотрим граф с матрицей смежности, приведенной в примере применения параллельного Алгоритма 1, v_S=0. Пусть 4 процесса составляют двумерный массив размера 2×2. Имеем: n_{adj}=7, R=2, C=2. Для того чтобы число $\frac{n_{adj}}{R\cdot C}$ было целым, добавим в граф фиктивную вершину; тогда n_{adj}=8, $\frac{n_{adj}}{R\cdot C}$=2. Рассмотрим транспонированную матрицу смежности:

$$A^T=\begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Разбиение опишем схемой:

$$\begin{array}{c} \\ \mathrm{Pr}_{1,1} \\ \mathrm{Pr}_{2,1} \\ \mathrm{Pr}_{1,2} \\ \mathrm{Pr}_{2,2} \end{array} \begin{array}{c} \begin{array}{cc} \mathrm{Pr}_{1,1}, \mathrm{Pr}_{2,1} & \mathrm{Pr}_{1,2}, \mathrm{Pr}_{2,2} \end{array} \\ \begin{pmatrix} A_{1,1}^{(1)} & A_{1,2}^{(1)} \\ A_{2,1}^{(1)} & A_{2,2}^{(1)} \\ A_{1,1}^{(2)} & A_{1,2}^{(2)} \\ A_{2,1}^{(2)} & A_{2,2}^{(2)} \end{pmatrix} \end{array}$$

или, в развернутом виде,

$$\begin{array}{cc} & \begin{array}{cccc} \mathrm{Pr}_{1,1} & & \mathrm{Pr}_{2,1} & \end{array} \quad \begin{array}{cccc} \mathrm{Pr}_{1,2} & & \mathrm{Pr}_{2,2} & \end{array} \\ v & \boxed{0\ 1} \quad \boxed{2\ 3} & & \boxed{4\ 5} \quad \boxed{6\ 7} \end{array}$$

$$\begin{array}{cc} \mathrm{Pr}_{1,1} \begin{matrix} \boxed{0} \\ \boxed{1} \end{matrix} & \begin{pmatrix} A_{1,1}^{(1)} \begin{matrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{matrix} & A_{1,2}^{(1)} \begin{matrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix} \\ A_{2,1}^{(1)} \begin{matrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{matrix} & A_{2,2}^{(1)} \begin{matrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix} \\ A_{1,1}^{(2)} \begin{matrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{matrix} & A_{1,2}^{(2)} \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{matrix} \\ A_{2,1}^{(2)} \begin{matrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{matrix} & A_{2,2}^{(2)} \begin{matrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix} \end{pmatrix} \\ \mathrm{Pr}_{2,1} \begin{matrix} \boxed{2} \\ \boxed{3} \end{matrix} & \\ \mathrm{Pr}_{1,2} \begin{matrix} \boxed{4} \\ \boxed{5} \end{matrix} & \\ \mathrm{Pr}_{2,2} \begin{matrix} \boxed{6} \\ \boxed{7} \end{matrix} & \end{array}.$$

Транспонированная матрица смежности A^T разбивается на четыре ($R\times C$=4) блочные строки и два (C=2) блочных столбца; блоки $A_{i,j}^{(1)}$ и $A_{i,j}^{(2)}$ принадлежат процессу $\mathrm{Pr}_{i,j}$. В каждом блоке две строки ($\frac{n_{adj}}{R\cdot C}$=2) и четыре столбца ($\frac{n_{adj}}{C}$=4). Один процесс владеет двумя блоками, т.е. владеет информацией о дугах, которые соответствуют ненулевым элементам (если такие элементы есть) этих блоков транспонированной матрицы смежности. Распределение вершин: процесс $\mathrm{Pr}_{i,j}$ владеет вершинами, соответствующими блочной строке с номером $(j-1)2+i$.

Перейдем непосредственно к применению параллельного Алгоритма 2.

Инициализация:

$\mathrm{Pr}_{1,1}$: $L(v_S)=L(0)=0,\ L(1)=\infty$,

$\mathrm{Pr}_{2,1}$: $L(2)=\infty,\ L(3)=\infty$,

$\mathrm{Pr}_{1,2}$: $L(4)=\infty,\ L(5)=\infty$,

$\mathrm{Pr}_{2,2}$: $L(6)=\infty,\ L(7)=\infty$.

Итерация l = 0:

Множества F:

$\mathrm{Pr}_{1,1}$: $F=\{0\}$,

$\mathrm{Pr}_{2,1}$, $\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: $F=\varnothing$.

Результаты обмена множествами F:

между процессами столбца 1:

$\mathrm{Pr}_{1,1}$, $\mathrm{Pr}_{2,1}$: $\overline{F}=\{0\}$,

между процессами столбца 2:

$\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: $\overline{F}=\varnothing$.

Множества N (N формируем из пар (v,u), где u – родитель v), N_q (N_q указаны только не пустые):

$\mathrm{Pr}_{1,1}$: $N=\{(1,0)\}$, $N_{1,1}=\{(1,0)\}$,

$\mathrm{Pr}_{2,1}$: $N=\{(3,0)\}$, $N_{2,1}=\{(3,0)\}$,

$\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: $N=\varnothing$.

Результаты обмена множествами N_q:

между процессами строки 1:

$\mathrm{Pr}_{1,1}$: $\overline{N}_{1,1}=\{(1,0)\}$, $\overline{N}=\{(1,0)\}$,

$\mathrm{Pr}_{1,2}$: $\overline{N}=\varnothing$,

между процессами строки 2:

$\mathrm{Pr}_{2,1}$: $\overline{N}_{2,1}=\{(3,0)\}$, $\overline{N}=\{(3,0)\}$,

$\mathrm{Pr}_{2,2}$: $\overline{N}=\varnothing$

(фактически обменов в данном случае не произошло).

Метки $L(v)$ и $P(v)$ для $v\in\overline{N}$ (если $L(v)=\infty$):

$\mathrm{Pr}_{1,1}$: $L(1)=1$, $P(1)=0$,

$\mathrm{Pr}_{2,1}$: $L(3)=1$, $P(3)=0$,

$\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: – .

Итерация l = 1:

Множества F:

$\mathrm{Pr}_{1,1}$: $F=\{1\}$,

$\mathrm{Pr}_{2,1}$: $F=\{3\}$,

$\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: $F=\varnothing$.

Результаты обмена множествами F:

между процессами столбца 1:

$\mathrm{Pr}_{1,1}$, $\mathrm{Pr}_{2,1}$: $\overline{F}=\{1, 3\}$,

между процессами столбца 2:

$\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: $\overline{F}=\varnothing$.

Множества N, N_q:

$\mathrm{Pr}_{1,1}$: $N=\{(4,1), (0,3)\}$, $N_{1,2}=\{(4,1)\}$, $N_{1,1}=\{(0,3)\}$,

$\mathrm{Pr}_{2,1}$: $N=\{(6,1), (2,3)\}$ ((v,u)=(6,3) в N не вошло, так как v=6 уже присутствует в паре (v,u)=(6,1)), $N_{2,2}=\{(6,1)\}$, $N_{2,1}=\{(2,3)\}$,

$\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: $N=\varnothing$.

Результаты обмена множествами N_q:

между процессами строки 1:

$\mathrm{Pr}_{1,1}$: $\overline{N}_{1,1}=\{(0,3)\}$, $\overline{N}=\{(0,3)\}$,

$\mathrm{Pr}_{1,2}$: $\overline{N}_{1,1}=\{(4,1)\}$, $\overline{N}=\{(4,1)\}$,

между процессами строки 2:

$\mathrm{Pr}_{2,1}$: $\overline{N}_{2,1}=\{(2,3)\}$, $\overline{N}=\{(2,3)\}$,

$\mathrm{Pr}_{2,2}$: $\overline{N}_{2,1}=\{(6,1)\}$, $\overline{N}=\{(6,1)\}$.

Метки $L(v)$ и $P(v)$ для $v\in\overline{N}$ (если $L(v)=\infty$):

$\mathrm{Pr}_{1,1}$: – (вершина 0 уже помечена: было $L(0)$=0),

$\mathrm{Pr}_{2,1}$: $L(2)$=2, $P(2)$=3,

$\mathrm{Pr}_{1,2}$: $L(4)$=2, $P(4)$=1,

$\mathrm{Pr}_{2,2}$: $L(6)$=2, $P(6)$=1.

Итерация l = 2:

Множества F:

$\mathrm{Pr}_{1,1}$: $F=\varnothing$,

$\mathrm{Pr}_{2,1}$: $F=\{2\}$,

$\mathrm{Pr}_{1,2}$: $F=\{4\}$,

$\mathrm{Pr}_{2,2}$: $F=\{6\}$.

Результаты обмена множествами F:

между процессами столбца 1:

$\mathrm{Pr}_{1,1}$, $\mathrm{Pr}_{2,1}$: $\overline{F}=\{2\}$,

между процессами столбца 2:

$\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: $\overline{F}=\{4, 6\}$.

Множества N, N_q:

$\mathrm{Pr}_{1,1}$: $N=\{(5,2)\}$, $N_{1,2}=\{(5,2)\}$,

$\mathrm{Pr}_{2,1}$: $N=\{(6,2)\}$, $N_{2,2}=\{(6,2)\}$,

$\mathrm{Pr}_{1,2}$: $N=\{(5,4),(4,6)\}$, $N_{1,2}=\{(5,4),(4,6)\}$,

$\mathrm{Pr}_{2,2}$: $N=\varnothing$.

Результаты обмена множествами N_q:

между процессами строки 1:

$\mathrm{Pr}_{1,1}$: $\overline{N}=\varnothing$,

$\mathrm{Pr}_{1,2}$: $\overline{N}_{1,1}=\{(5,2)\}$, $\overline{N}_{1,2}=\{(5,4),(4,6)\}$, $\overline{N}=\{(5,2),(4,6)\}$

((v,u)= (5,4) в $\overline{N}$ не вошло, так как v=5 уже присутствует в паре (v,u)=(5,2)),

между процессами строки 2:

$\mathrm{Pr}_{2,1}$: $\overline{N}=\varnothing$,

$\mathrm{Pr}_{2,2}$: $\overline{N}_{2,1}=\{(6,2)\}$, $\overline{N}=\{(6,2)\}$.

Метки $L(v)$ и $P(v)$ для $v\in\overline{N}$ (если $L(v)=\infty$):

$\mathrm{Pr}_{1,1}$: – ,

$\mathrm{Pr}_{2,1}$: – ,

$\mathrm{Pr}_{1,2}$: $L(5)=3$, $P(5)=2$, (вершина 4 уже помечена),

$\mathrm{Pr}_{2,2}$: – (вершина 6 уже помечена).

Итерация l = 3:

Множества F:

$\mathrm{Pr}_{1,1}$: $F=\varnothing$,

$\mathrm{Pr}_{2,1}$: $F=\varnothing$,

$\mathrm{Pr}_{1,2}$: $F=\{5\}$,

$\mathrm{Pr}_{2,2}$: $F=\varnothing$.

Результаты обмена множествами F:

между процессами столбца 1:

$\mathrm{Pr}_{1,1}$, $\mathrm{Pr}_{2,1}$: $\overline{F}=\varnothing$,

между процессами столбца 2:

$\mathrm{Pr}_{1,2}$, $\mathrm{Pr}_{2,2}$: $\overline{F}=\{5\}$.

Множества N, N_q:

$\mathrm{Pr}_{1,1}$: $N=\varnothing$,

$\mathrm{Pr}_{2,1}$: $N=\varnothing$,

$\mathrm{Pr}_{1,2}$: $N=\varnothing$,

$\mathrm{Pr}_{2,2}$: $N=\{(2,5)\}$, $N_{2,1}=\{(2,5)\}$.

Результаты обмена множествами N_q:

между процессами строки 1:

$\mathrm{Pr}_{1,1}$: $\overline{N}=\varnothing$,

$\mathrm{Pr}_{1,2}$: $\overline{N}=\varnothing$,

между процессами строки 2:

$\mathrm{Pr}_{2,1}$: $\overline{N}_{2,2}=\{(2,5)\}$, $\overline{N}=\{(2,5)\}$,

$\mathrm{Pr}_{2,2}$: $\overline{N}=\varnothing$.

Метки $L(v)$ и $P(v)$ для $v\in\overline{N}$ (если $L(v)=\infty$):

$\mathrm{Pr}_{1,1}$: – ,

$\mathrm{Pr}_{2,1}$: – (вершина 2 уже помечена),

$\mathrm{Pr}_{1,2}$: – ,

$\mathrm{Pr}_{2,2}$: – .

Итерация l = 4:

Множества F: для всех процессов $F=\varnothing$,

все вычислительные процессы завершаются.

Другие параллельные алгоритмы

Параллельный 2D алгоритм, использующий умножение матрицы смежности на вектор (параллельный алгебраический алгоритм поиска в ширину в графе) [3, 4]**.** Параллельный 2D алгоритм работы [2] ориентирован на графы малого диаметра с однородной (равномерной) степенью распределения (low-diameter graphs with uniform degree distribution). Параллельный 2D алгоритм, предложенный в работах [3, 4], ориентирован на графы малого диаметра с неравномерной степенью распределения (low-diameter graphs with skewed degree distribution) (именно такие графы рассматриваются в тесте BFS проекта Graph 500). Параллельный 2D алгоритм [3, 4] для реализации на компьютере с распределенной памятью основан на так называемом «шахматном» разбиении матрицы смежности.

Параллельные алгоритмы для реализации на многоядерном CPU, на GPU, на комбинировании GPU и многоядерного CPU [5–8]**.** Рассматриваются, в частности, оптимизации для графов с разреженными матрицами смежности и графов малого диаметра.

Сверху-вниз и снизу-вверх стратегии поиска. Рассмотренный алгоритм поиска в ширину реализует традиционный подход, так называемую сверху-вниз стратегию поиска (top-down search strategy): на очередной итерации ищутся соседи каждой просматриваемой на этой итерации вершины. Существует еще снизу-вверх стратегия поиска (bottom-up search strategy): на очередной итерации каждая непомеченная вершина проверяет

вершины, из которых она доступна, пытаясь найти среди них родителя, т.е. вершину, помеченную на предыдущей итерации.

Снизу-вверх стратегия эффективна только тогда, когда на итерациях просматривается много вершин. Поэтому в начале и конце алгоритма поиска в ширину следует применять сверху-вниз стратегию, а для средних итераций следует применять снизу-вверх стратегию. Параллельный алгоритм, реализующий такой подход, предложены в работах [9, 10]. Вычислительные эксперименты показали, что гибридное применение двух стратегий дает значительное повышение производительности.

В работе [3] представлены результаты вычислительных экспериментов, в которых сравнивались параллельные алгоритмы с применением 1D и 2D разбиения и сверху-вниз стратегией поиска, а также модифицированные (не просто MPI-реализации, а MPI+OpenMP-реализации) алгоритмы. Эксперименты показали, что параллельные алгоритмы с применением 2D разбиения допускают, в зависимости от используемого суперкомпьютера, получение как лучшей, так и худшей, по сравнению со случаем 1D разбиения, производительности (время на коммуникации всегда лучше).

Строчный формат хранения разреженных матриц

В рассмотренных примерах фигурировали плотные матрицы и векторы. В действительности они являются разреженными и на практике используется разреженный строчный формат (CSR – Compressed Sparse Rows) хранения разреженных матриц [11]. CSR предъявляет минимальные требования к памяти и обеспечивает эффективный доступ к строкам матрицы; доступ к столбцам затруднен, поэтому предпочтительно использовать эту схему хранения в тех алгоритмах, в которых преобладают строчные операции.

В соответствии с CSR-схемой, для хранения матрицы A требуется три одномерных массива:

- массив ненулевых элементов матрицы A, в котором они перечислены по строкам от первой до последней (обозначим его как values);
- массив номеров столбцов для соответствующих элементов массива values (обозначим его как cols);
- массив указателей позиций, с которых начинается описание очередной строки (обозначим его pointer). Описание k-й строки хранится в позициях с pointer[k]-й по (pointer[k+1]–1)-ю массивов values и cols. Если pointer[k]=pointer[k+1], то k-я строка пустая. Если матрица A состоит из n строк, то длина массива pointer равна n+1, причем (n+1)-й элемент массива на 1 превышает количество элементов массива values (и массива cols).

Рассмотрим представление в разреженном строчном формате матрицы

$$A^T = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

из примера применения параллельного алгоритма 2:

values=(1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1);

cols=(4, 1, 4, 6, 1, 2, 7, 3, 5, 2, 3, 4);

pointer=(1, 2, 3, 5, 6, 8, 10, 13, 13).

СПИСОК ЛИТЕРАТУРЫ

1. Котов В.М., Соболевская Е.П., Толстиков А.А. Алгоритмы и структуры данных. Учебное пособие. Минск: БГУ, 2011. 267 с.

2. Yoo A., Chow E., Henderson K., McLendon W., Hendrickson B., Catalyurek U.V: A scalable distributed parallel breadth-first search algorithm on BlueGene/L // Proceedings of the ACM/IEEE supercomputing 2005 conference. November 2005.

3. Buluc A., Madduri K. Parallel breadth-first search on distributed memory systems // Proceedings of the ACM/IEEE supercomputing 2011 conference. November 2011.

4. Buluc A., Madduri K. Graph partitioning for scalable distributed graph computations // 10th DIMACS implementation challenge, graph partitioning and graph clustering. February 13–14, 2012 Atlanta, Georgia.

5. Agarwal V., Petrini F., Pasetto D., Bader D. Scalable graph exploration on multicore processors // Proceedings of the ACM/IEEE supercomputing 2010 conference. New Orleans, LA, November 13–19, 2010.

6. Luo L., Wong M., Hwu W. An effective GPU implementation of breadth-first search // Proceedings of the design automation 2010 conference. June 2010.

7. Hong S., Kim S., Oguntebi T., Olukotun K. Accelerating CUDA graph algorithms at maximum warp // Proceedings of the 16th ACM symposium on principles and practice of parallel programming, 2011.

8. Hong S., Kim S., Oguntebi T., Olukotun K. Efficient parallel graph exploration on multi-core CPU and GPU // Proceedings of the parallel architectures and compilation techniques 2011 conference.

9. Beamer S., Buluc A., Asanovic K., Patterson D. Distributed memory breadth-first search revisited: enabling bottom-up search // IEEE international symposium on parallel and distributed processing. May 20–24, 2013. Cambridge, MA, USA.

10. Черноскутов М.А. Параллельная высокопроизводительная обработка графов // Международная научная конференция «Параллельные

вычислительные технологии» (ПаВТ’2016): Труды международной научной конференции (Архангельск, 28 марта – 1 апреля 2016 г.). С. 736–742. (Доклад в электронном виде)

11. Писсанецки С. Технология разреженных матриц. М.: Мир. 1988. 411 с.